3 Pipelines	8 Deliverables	$700 Prize Pool	2M+ Tokens Min

TABLE OF CONTENTS

1. Problem Statement & Goals

Why this hackathon exists and what winning looks like

LLMs consume thousands of tokens to answer complex questions. As deployment scale grows, companies pay more, hit context limits faster, and face rising latency. **Basic RAG** (vector embeddings + LLM) helps, but can only retrieve *similar* chunks — it cannot *reason* across entity relationships spanning multiple documents. **GraphRAG** solves this by organising data into an entity-relationship graph, performing multi-hop traversal, and delivering a focused, structured context to the LLM.

The headline metric: Token reduction with maintained (or improved) answer accuracy.

■ IMPORTANT — Token reduction alone is NOT a win. A pipeline that cuts tokens by 70% but loses 20% accuracy is a **regression**, not a victory. Both dimensions must move in the right direction.

Judging Criteria

Criteria	Weight	What Judges Look For
Token Reduction	30%	% drop in tokens + cost vs Basic RAG baseline
Answer Accuracy	30%	LLM-as-a-Judge (PASS/FAIL) + BERTScore
Performance	20%	Latency, throughput, end-to-end efficiency
Engineering & Storytelling	20%	Clean code, working dashboard, demo video, blog

Bonus Points (automatically rewarded in scoring)

Judge Bonus

- **LLM-as-a-Judge pass rate:** ≥ 90% PASS across all test queries
- **Hitting BOTH thresholds:** unlocks maximum bonus multiplier

BERTScore Bonus

- **BERTScore F1 rescaled:** ≥ 0.55 OR raw variant ≥ 0.88
- **Strategy tip:** Tune iteratively — chunk size, hop depth, prompt template

Two-Round Structure

Round	Who	Dataset Size	LLM Credits	Outcome
Round 1 — Open Hackathon	All teams globally	≥ 2 M tokens	Your own free tier (Gemini, Ollama…)	Top 10 advance
Round 2 — Production Showcase	Top 10 only	50–100 M tokens	$50 Gemini API credits per team (provided)	Winners announced

2. Architecture Overview

Three parallel pipelines on one shared corpus — one dashboard to compare them all

Every pipeline ingests the **same corpus**, answers the **same question set**, and is measured against the **same evaluation harness**. The only variable is the retrieval strategy.

Layer	Pipeline 1 LLM-Only	Pipeline 2 Basic RAG	Pipeline 3 GraphRAG
Ingestion	Raw text only	Chunk + embed	Chunk + extract entities + build graph
Index	—	Vector store (FAISS / pgvector)	TigerGraph knowledge graph
Retrieval	None	Top-k cosine similarity	Hybrid / Community / Sibling retriever
Context	Empty / minimal	k flat chunks	Structured paths + summaries
Generation	LLM answers from params	LLM answers from chunks	LLM answers from graph context
Expected tokens	**Highest**	Medium	**Lowest** (target)

Shared Infrastructure

- Single corpus loader — all three pipelines read identical files
- Shared tokeniser — tiktoken cl100k_base for all token counts
- Shared question bank — minimum 20 questions across 3 strata
- Shared LLM provider — same model, same temperature
- Evaluation harness — BERTScore + HuggingFace Judge on same answers
- Dashboard — React / Streamlit aggregates all pipeline metrics
- Logging — structured JSON per query (tokens, latency, cost)
- GitHub mono-repo — all three pipelines in one codebase

3. Environment & Repository Setup

Step-by-step from zero to a running TigerGraph GraphRAG environment

Step 1 — Choose Your TigerGraph Environment

Option A — TigerGraph Savanna (RECOMMENDED)

- Web-based, zero local installation required
- Sign up at tgcloud.io → get $60 in free credits instantly
- More credits available on request from TigerGraph team
- Best choice if you want to focus on benchmarking, not ops

Option B — Community Edition (Local)

- Free, runs locally via Docker
- Download from dl.tigergraph.com
- Best if you need offline or custom infra control
- Requires Docker Desktop 4.x+, 8 GB RAM minimum

Step 2 — Clone the TigerGraph GraphRAG Repository

```
# Clone the official TigerGraph GraphRAG repo
git clone https://github.com/tigergraph/graphrag.git
cd graphrag

# Install Python dependencies
pip install -r requirements.txt

# Copy environment template and fill in your API keys
cp .env.example .env
# Edit .env: set TG_HOST, TG_USERNAME, TG_PASSWORD, TG_GRAPH_NAME,
#             OPENAI_API_KEY (or GOOGLE_API_KEY for Gemini)
```

Clone & install TigerGraph GraphRAG repo

Step 3 — Get an LLM API Key

Provider	Free Tier	Recommended Model	Notes
Google Gemini	Yes — generous free tier	gemini-1.5-flash	Best for hackathon scale; used in Round 2
Ollama (local)	Completely free	llama3.2 / mistral	No API costs; needs 16 GB RAM
OpenAI	Limited ($5 trial)	gpt-4o-mini	Best accuracy; watch costs
Anthropic Claude	API credits needed	claude-haiku-4-5	Good quality/cost balance

Step 4 — Deploy GraphRAG Service via Docker

```
# From inside the graphrag/ directory:
docker compose up -d

# Verify the service is running
curl http://localhost:8000/health
# Expected: {"status": "ok"}

# Ingest your dataset (see Section 4 for dataset prep)
curl -X POST http://localhost:8000/ingest \
  -H 'Content-Type: application/json' \
  -d '{"source": "./data/corpus/", "graph_name": "my_graph"}'
```

Deploy and verify the GraphRAG Docker service

Step 5 — (Optional) MCP Integration for AI-Assisted Development

■ Connect TigerGraph directly to Cursor or VS Code Copilot via MCP (github.com/tigergraph/tigergraph-mcp). This lets you query and build with natural language — no GSQL boilerplate. Highly recommended if you are new to TigerGraph.

4. Dataset Selection & Preparation

Choosing the right corpus is the most important non-code decision you will make

GraphRAG performs best when data has **natural relational density** — entities (people, places, events, concepts) that cross-reference each other across documents. A corpus of isolated monologs with no named entities will show almost no GraphRAG advantage.

Recommended Dataset Domains

Domain	Why It Works for GraphRAG	Source	Approx Size
Scientific papers (arXiv)	Authors, citations, concepts link richly across papers	arxiv.org bulk download	~5M tokens / 1000 papers
Legal documents (court cases)	Cases cite cases; judges, laws, entities recur	CourtListener (courtlistener.com)	~4M tokens / 500 cases
Medical literature (PubMed)	Genes, drugs, diseases form natural entity graph	PubMed Central Open Access	~6M tokens / 800 papers
Wikipedia dump (topic slice)	Rich hyperlinks = natural entity graph	dumps.wikimedia.org	~10M tokens / 5000 articles
News archives (news articles)	Events, people, orgs link across stories	CC-News, GDELT	~3M tokens / 3000 articles

■ Dataset must be public domain or properly licensed. Do NOT use copyrighted books, paywalled papers, or proprietary content. Always document your license source.

Dataset Preparation Algorithm

```
ALGORITHM: PrepareCorpus(raw_files)
═══════════════════════════════════════════════════════════════
Input : List of raw text files (PDF / TXT / JSON)
Output: Normalised chunks with stable IDs, metadata JSON

1.  LOAD each file → extract text (pypdf / BeautifulSoup)
2.  NORMALISE:
     a. Strip HTML tags, headers/footers, page numbers
     b. Normalise Unicode (unicodedata.normalize NFC)
     c. Collapse whitespace; remove null bytes
3.  DEDUPLICATE using MinHash LSH (datasketch) — remove near-duplicates
4.  COUNT total tokens using tiktoken cl100k_base
     → Abort if total < 2,000,000 tokens (Round 1 minimum)
5.  CHUNK:
     chunk_size  = 384 tokens   (search range: 256 / 384 / 512)
     overlap     = 64  tokens   (search range: 32 / 64 / 96)
     → RecursiveCharacterTextSplitter from langchain
6.  ASSIGN each chunk a stable UUID based on (source_file, position)
7.  WRITE chunks to ./data/chunks.jsonl
     Each record: {id, source, text, token_count, metadata}
8.  VERIFY token budget:
     total = sum(chunk.token_count for chunk in chunks)
     assert total >= 2_000_000
═══════════════════════════════════════════════════════════════
```

Corpus preparation algorithm — shared by all three pipelines

Question Bank Construction

Create at least **20 test questions** — stratified across three types. This stratification is critical: it exposes where each pipeline wins and loses, giving judges richer evidence.

Question Type	Count (min)	Example	Expected GraphRAG Win?
Local / Entity-centric	7	What is [Entity X]'s role in [Event Y]?	Moderate
Global / Corpus synthesis	7	What are the main themes across all documents?	Strong
Multi-hop / Relational	6	How did A influence B which led to C?	Strongest

5. Pipeline 1 — LLM-Only Baseline

No retrieval. Your worst-case cost baseline. Establishes the token ceiling.

This pipeline sends the user query *directly* to the LLM with only a system prompt and the query. No corpus text is retrieved. It represents the maximum token overhead scenario when the LLM must rely solely on parametric memory.

Implementation Algorithm

```
ALGORITHM: LLMOnlyQuery(query, llm_client)
═══════════════════════════════════════════════════════════════
Input : query (string), llm_client
Output: {answer, prompt_tokens, completion_tokens, latency_ms, cost}

1.  BUILD prompt:
     system_prompt = "Answer the following question using your knowledge.
                      If unsure, say so. Be concise."
     messages = [{role:system, content:system_prompt},
                 {role:user,   content:query}]

2.  START timer  t0 = time.perf_counter()

3.  CALL llm_client.chat.completions.create(
          model    = config.model,
          messages = messages,
          temperature = 0.0    # deterministic for reproducibility
     )

4.  END timer  latency_ms = (time.perf_counter() - t0) * 1000

5.  EXTRACT:
     answer           = response.choices[0].message.content
     prompt_tokens    = response.usage.prompt_tokens
     completion_tokens= response.usage.completion_tokens

6.  CALCULATE:
     cost = (prompt_tokens * price_in) + (completion_tokens * price_out)
     # price_in/out from your provider's pricing page

7.  RETURN {answer, prompt_tokens, completion_tokens, latency_ms, cost}
═══════════════════════════════════════════════════════════════
```

Pipeline 1 — LLM-Only query algorithm

✓ This pipeline will almost always have the HIGHEST token count and cost. That is expected. Its role is to set the upper bound that Basic RAG and GraphRAG must beat.

6. Pipeline 2 — Basic RAG (Vector)

The industry standard. Dense retrieval + LLM. Your quality & cost reference point.

Basic RAG embeds all chunks into a vector store, then at query time retrieves the top-k most similar chunks by cosine similarity and stuffs them into the LLM prompt. This is the baseline GraphRAG must beat on **every metric that matters**.

Step-by-Step Implementation

Step 6a — Build the Vector Index

```
ALGORITHM: BuildVectorIndex(chunks)
═══════════════════════════════════════════════════════════════
Input : List of chunks from Section 4
Output: FAISS index  +  chunk lookup dict

1.  LOAD embedding model:
     model = SentenceTransformer('all-MiniLM-L6-v2')  # free, fast
     # Alternative: text-embedding-3-small (OpenAI) for higher quality

2.  BATCH-ENCODE all chunks:
     embeddings = model.encode(
         [c['text'] for c in chunks],
         batch_size=256, show_progress_bar=True
     )   # shape: [N, 384]

3.  BUILD FAISS index:
     dim   = embeddings.shape[1]
     index = faiss.IndexFlatIP(dim)   # Inner Product ≡ cosine on unit vecs
     faiss.normalize_L2(embeddings)   # normalise to unit vectors
     index.add(embeddings)

4.  SAVE:
     faiss.write_index(index, './data/vector.index')
     json.dump(chunks, open('./data/chunks_meta.json','w'))
═══════════════════════════════════════════════════════════════
```

Build FAISS vector index over corpus chunks

Step 6b — Query the Basic RAG Pipeline

```
ALGORITHM: BasicRAGQuery(query, index, chunks, llm_client, k=8)
═══════════════════════════════════════════════════════════════
Input : query string, FAISS index, chunk list, LLM client, k
Output: {answer, prompt_tokens, completion_tokens, latency_ms, cost}

1.  ENCODE query:
     q_vec = model.encode([query])  # shape [1, 384]
     faiss.normalize_L2(q_vec)

2.  RETRIEVE top-k chunks:
     distances, indices = index.search(q_vec, k)
     retrieved = [chunks[i] for i in indices[0]]

3.  DEDUPLICATE by chunk ID (guard against near-duplicate chunks)

4.  BUILD context string:
     context = '\n\n'.join([
         f'[Chunk {i+1}]\n{c["text"]}'
         for i, c in enumerate(retrieved)
     ])

5.  BUILD prompt:
     system = 'Answer the question using ONLY the provided context.
               If the context is insufficient, say so.'
     user   = f'Context:\n{context}\n\nQuestion: {query}'

6.  CALL LLM and measure tokens + latency (same as Pipeline 1 Step 3-6)

7.  RETURN {answer, prompt_tokens, completion_tokens, latency_ms, cost}
═══════════════════════════════════════════════════════════════
```

Pipeline 2 — Basic RAG query algorithm (k=8 starting point)

Key Parameters to Track

Parameter	Starting Value	Effect if Increased	Effect if Decreased
k (top_k chunks retrieved)	8	More recall, more tokens, slower	Fewer tokens, may miss facts
Chunk size	384 tokens	Richer context per chunk, fewer chunks	More chunks, finer granularity
Chunk overlap	64 tokens	Better boundary coverage, slight redundancy	Faster indexing, possible gaps
Rerank (optional)	Off → try ColBERT	Better relevance, fewer wasted chunks	Lower quality retrieval

7. Pipeline 3 — GraphRAG (TigerGraph)

Entity graph + multi-hop reasoning. Two valid paths — both judged on outcomes.

TigerGraph GraphRAG converts your document corpus into a **knowledge graph**: entities become vertices, relationships become edges, and communities of related entities get pre-computed summary reports. At query time, the system traverses this graph instead of scanning all chunks, producing a compact, structured context for the LLM.

Choose Your Path

Path A — Use As-Is	Path B — Customise
Deploy GraphRAG via Docker, ingest data, call REST/Python APIs. Treat it as a black box. Focus on benchmarking and storytelling.	Deploy repo, then tune parameters, swap retrievers, modify prompt templates, extend the schema, or add your own routing logic.
Best for: teams focused on the comparison dashboard and results narrative.	*Best for: teams who want an engineering challenge and maximum scoring potential.*

7a. Path A — GraphRAG As-Is (Detailed Steps)

```
# 1. Ingest corpus into TigerGraph GraphRAG
import requests, json

GRAPHRAG_URL = 'http://localhost:8000'

# Upload corpus files
resp = requests.post(f'{GRAPHRAG_URL}/ingest', json={
    'source_dir': './data/corpus/',
    'graph_name': 'hackathon_graph',
    'chunk_size': 384,
    'chunk_overlap': 64
})
print(resp.json())   # {status: 'ingested', entities: N, edges: M}

# 2. Query via REST API (Local search — entity-centric)
def graphrag_local_query(query):
    resp = requests.post(f'{GRAPHRAG_URL}/query/local', json={
        'query': query,
        'top_k': 5,          # seed entities to retrieve
        'num_hops': 2,       # graph expansion depth
    })
    return resp.json()   # {answer, context, tokens_used, latency_ms}

# 3. Query via REST API (Global search — community summaries)
def graphrag_global_query(query):
    resp = requests.post(f'{GRAPHRAG_URL}/query/global', json={
        'query': query,
        'community_level': 2,   # 1=coarse, 3=fine
    })
    return resp.json()
```

Path A — Ingest corpus and query via TigerGraph GraphRAG REST API

7b. Path B — Customisation Options

Customisation	Parameter / Code Change	Expected Effect
Retriever type	Switch: Hybrid Search ↔ Community ↔ Sibling	Different precision/recall trade-offs
Hop depth	num_hops: 1 → 2 → 3	Deeper reasoning, but more tokens
Community level	community_level: 1 → 2 → 3	Coarser → finer community reports
Seed size	top_k: 3 → 5 → 8	Broader starting context
Prompt template	Edit prompt_templates/local.txt	Controls LLM reasoning style
Schema extension	Add vertex/edge types in schema.gsql	Capture domain-specific relations
Token budget	max_context_tokens: 2000 → 3000 → 4000	Hard cap on prompt size

8. Graph Construction Algorithms

How TigerGraph GraphRAG builds its knowledge graph — and how to tune it

8a. Entity Extraction

```
ALGORITHM: ExtractEntities(chunks)
═══════════════════════════════════════════════════════════════════
Input : List of text chunks
Output: List of (entity_name, entity_type, chunk_id, context)

1.  FOR each chunk c in chunks:
     a. SEND to LLM with NER prompt:
        'Extract all named entities (PERSON, ORG, LOCATION, CONCEPT,
         EVENT, DATE) from the following text. Return JSON list.
         Text: {c.text}'
     b. PARSE JSON response → list of {name, type, mentions}
     c. NORMALISE entity names (lower-case, remove punctuation)
     d. RESOLVE coreferences (e.g. 'He' → previous PERSON entity)

2.  DEDUPLICATE entities by normalised name + type

3.  ASSIGN stable vertex IDs: entity_id = hash(name + type)

4.  EXTRACT relationships between co-occurring entities:
     FOR each pair (e1, e2) in same sentence:
         relation_type = classify_relation(e1, e2, sentence)
         # e.g. WORKS_AT, LOCATED_IN, MENTIONED_WITH, CAUSED_BY

5.  LOAD into TigerGraph:
     gsql: UPSERT VERTEX Entity(id, name, type, description)
     gsql: UPSERT EDGE Relationship(src, tgt, type, weight, chunk_id)
═══════════════════════════════════════════════════════════════════
```

Entity extraction and graph loading algorithm

8b. Community Detection (Leiden Algorithm)

```
ALGORITHM: BuildCommunities(entity_graph)
═══════════════════════════════════════════════════════════════
Input : Entity graph G = (V, E) loaded in TigerGraph
Output: Hierarchical community structure + summary reports

1.  RUN Leiden algorithm on G:
     communities = leiden(G,
         resolution=1.0,
         n_iterations=10
     )
     # Leiden produces hierarchical partitions at multiple levels

2.  FOR each community C at each level L:
     a. COLLECT all entities and their descriptions in C
     b. COLLECT all edges (relationships) within C
     c. GENERATE community summary via LLM:
        prompt = 'Summarise the following entities and their relationships
                  into a coherent paragraph. Focus on the main theme.
                  Entities: {entity_list}
                  Relationships: {edge_list}'
     d. STORE summary as CommunityReport vertex in TigerGraph

3.  ASSIGN community_level attribute to each entity vertex

4.  INDEX community reports for fast global query retrieval
═══════════════════════════════════════════════════════════════
```

Community detection and summary generation (Leiden-based)

8c. PathRAG-Inspired Context Pruning (Literature-Backed)

From the literature (PathRAG, AAAI 2026): the most common GraphRAG failure is **over-retrieval** — too many weak relational paths inflate the prompt without adding reasoning value. Apply flow-based pruning before prompt assembly.

```
ALGORITHM: PruneGraphContext(subgraph, query, max_tokens=3000)
═══════════════════════════════════════════════════════════════
Input : Retrieved subgraph, query, token budget
Output: Pruned set of relational paths for prompt assembly

1.  SCORE each path P = [e1 → r1 → e2 → r2 → ... → en] by:
     relevance(P) = cosine_sim(query_vec, path_text_vec)
     flow_score(P)= min(edge.weight for edge in P)   # weakest link
     distance_penalty(P) = 1 / (1 + len(P))         # prefer short paths
     score(P) = relevance * flow_score * distance_penalty

2.  SORT paths by score DESC

3.  GREEDILY SELECT paths until token budget exhausted:
     selected = []
     token_used = 0
     FOR path in sorted_paths:
         path_tokens = count_tokens(format_path(path))
         IF token_used + path_tokens <= max_tokens:
             selected.append(path)
             token_used += path_tokens
         ELSE:
             BREAK

4.  FORMAT selected paths as structured context:
     context = '\n'.join([
         f'{p.source} --[{p.relation}]--> {p.target}  ({p.evidence})'
         for p in selected
     ])

5.  RETURN context (guaranteed within token budget)
═══════════════════════════════════════════════════════════════
```

PathRAG-inspired flow-based context pruning

9. Query Routing & Retrieval Strategy

Route each query to the right retrieval mode — the key to both accuracy and token efficiency

Not every query needs full graph traversal. Routing is the mechanism that makes GraphRAG cost-effective: simple fact lookups use the cheaper local/dense path; global synthesis and multi-hop reasoning activate the graph. This approach is backed by EA-GraphRAG (arXiv 2026) and the TigerGraph GraphRAG's own Local / Global / DRIFT search modes.

Query Routing Algorithm

```
ALGORITHM: RouteAndAnswer(query, pipelines, config)
═══════════════════════════════════════════════════════════════
Input : query, {dense_rag, graphrag_local, graphrag_global}, config
Output: {answer, tokens, latency, cost, mode_used}

1.  CLASSIFY query complexity:
     features = {
         'has_multi_entity': count_named_entities(query) >= 2,
         'has_causal_words': any(w in query for w in
                             ['how did','why','caused','led to','resulted']),
         'has_comparison':   any(w in query for w in
                             ['compare','difference','vs','versus']),
         'is_global':        any(w in query for w in
                             ['all','overall','main theme','summarise',
                              'across all','general']),
         'word_count':       len(query.split())
     }

2.  SCORE = weighted_sum(features, weights=config.routing_weights)
     # weights are tunable on your dev question set

3.  ROUTE:
     IF features['is_global']:
         mode = 'graphrag_global'    # community summary map-reduce
     ELIF SCORE >= config.graph_threshold:
         mode = 'graphrag_local'     # entity + multi-hop traversal
     ELSE:
         mode = 'basic_rag'          # cheap vector similarity

4.  EXECUTE selected pipeline.query(query)

5.  IF answer quality low (judge_score < 0.5) AND mode != 'graphrag_global':
         # Fallback: escalate to next mode
         mode = escalate(mode)
         answer = execute(mode, query)

6.  RETURN {answer, tokens, latency, cost, mode_used: mode}
═══════════════════════════════════════════════════════════════
```

Adaptive query routing algorithm

TigerGraph GraphRAG Search Modes Reference

Mode	TigerGraph API Endpoint	Best For	Expected Token Usage
Local Search	POST /query/local	Entity-centric, fact lookup, named entity questions	Low–Medium

Mode	TigerGraph API Endpoint	Best For	Expected Token Usage
Global Search	POST /query/global	Corpus-wide synthesis, themes, summaries	Medium (map-reduce)
DRIFT Search	POST /query/drift	Multi-hop, evolving narratives, causal chains	Medium
Hybrid Search	POST /query/hybrid	Mixed queries — combines vector + graph signals	Medium
Sibling Search	POST /query/sibling	Find related entities at same graph level	Low
Community	POST /query/community	Cluster-level insights, topic groupings	Low (pre-computed)

10. Comparison Dashboard

One query in → three pipelines run → metrics side-by-side. The heart of your submission.

The dashboard must be **interactive**: type a query, click Run, and see all three pipeline responses with metrics displayed simultaneously. This is one of the eight required deliverables and counts toward Engineering & Storytelling (20% weight).

Recommended Stack (choose one)

Option A — Streamlit (Fastest to Build)

- pip install streamlit
- ~100 lines of Python for a functional dashboard
- Easy deployment to Streamlit Cloud (free)
- Built-in session state for query history

Option B — React + FastAPI (More Professional)

- FastAPI backend exposes /run_all_pipelines endpoint
- React frontend with Recharts for metric visualisation
- Cleaner demo video appearance
- More engineering effort (~300 lines total)

Dashboard Layout Specification

Core Dashboard Code Snippet (Streamlit)

```
import streamlit as st, time, json
from pipelines import llm_only, basic_rag, graphrag

st.title('■ GraphRAG Inference Benchmark Dashboard')
query = st.text_input('Enter your question:')

if st.button('■ Run All Pipelines') and query:
    cols = st.columns(3)
    labels = ['LLM-Only', 'Basic RAG', 'GraphRAG']
    runners = [llm_only.query, basic_rag.query, graphrag.query]

    results = []
    for col, label, runner in zip(cols, labels, runners):
        with col:
            with st.spinner(f'Running {label}...'):
                r = runner(query)
            st.subheader(label)
            st.write(r['answer'])
            st.metric('Tokens',   r['prompt_tokens'] + r['completion_tokens'])
            st.metric('Latency',  f"{r['latency_ms']:.0f} ms")
            st.metric('Cost',     f"${r['cost']:.4f}")
            results.append(r)

    # Token reduction vs Basic RAG
    base_tok = results[1]['prompt_tokens'] + results[1]['completion_tokens']
    graph_tok= results[2]['prompt_tokens'] + results[2]['completion_tokens']
    reduction = (base_tok - graph_tok) / base_tok * 100
    st.success(f'GraphRAG token reduction vs Basic RAG: {reduction:.1f}%')
```

Streamlit comparison dashboard — minimal working implementation

11. Evaluation Framework

LLM-as-a-Judge + BERTScore — targeting all bonus thresholds

■ The evaluation framework is how you PROVE GraphRAG wins. Use the EXACT methods specified in the hackathon brief: Hugging Face hosted LLM judge (PASS/FAIL) + BERTScore. Bonus thresholds: Judge ≥ 90% pass rate AND BERTScore F1 rescaled ≥ 0.55 (raw ≥ 0.88).

11a. LLM-as-a-Judge (Hugging Face PASS/FAIL)

```
from huggingface_hub import InferenceClient

JUDGE_MODEL = 'mistralai/Mistral-7B-Instruct-v0.2'  # Free on HuggingFace
client = InferenceClient(model=JUDGE_MODEL)

JUDGE_PROMPT = '''
You are an objective evaluator. Given a question, a reference answer, and a
candidate answer, determine if the candidate answer is correct and complete.
Reply with ONLY one word: PASS or FAIL.

Question:          {question}
Reference Answer:  {reference}
Candidate Answer:  {candidate}

Verdict:'''

def judge_answer(question, reference, candidate):
    prompt = JUDGE_PROMPT.format(
        question=question, reference=reference, candidate=candidate
    )
    # Randomise order to reduce position bias (literature recommendation)
    response = client.text_generation(prompt, max_new_tokens=5)
    verdict = response.strip().upper()
    return 1 if 'PASS' in verdict else 0

def evaluate_pipeline_judge(pipeline_results, qa_pairs):
    scores = []
    for r, qa in zip(pipeline_results, qa_pairs):
        score = judge_answer(qa['question'], qa['reference'], r['answer'])
        scores.append(score)
    pass_rate = sum(scores) / len(scores)
    print(f'LLM-as-a-Judge pass rate: {pass_rate:.2%}')
    print(f'Bonus threshold (>=90%): {"✓ MET" if pass_rate >= 0.90 else "✗ NOT MET"}')
    return pass_rate, scores
```

LLM-as-a-Judge evaluation using HuggingFace Inference API

11b. BERTScore Evaluation

```
from bert_score import score as bert_score_fn

def evaluate_pipeline_bertscore(pipeline_results, qa_pairs):
    candidates  = [r['answer']         for r in pipeline_results]
    references  = [qa['reference']      for qa in qa_pairs]

    P, R, F1 = bert_score_fn(
        candidates, references,
        lang='en',
        model_type='microsoft/deberta-xlarge-mnli',   # best for English
        rescale_with_baseline=True   # produces F1 rescaled in [0, 1]
    )

    f1_rescaled = F1.mean().item()
    f1_raw      = bert_score_fn(
        candidates, references, lang='en',
        rescale_with_baseline=False
    )[2].mean().item()

    print(f'BERTScore F1 (rescaled): {f1_rescaled:.4f}  ',
          f'(>= 0.55: {"✓ BONUS" if f1_rescaled >= 0.55 else "✗"})')
    print(f'BERTScore F1 (raw):      {f1_raw:.4f}  ',
          f'(>= 0.88: {"✓ BONUS" if f1_raw >= 0.88 else "✗"})')
    return f1_rescaled, f1_raw
```

BERTScore evaluation — rescaled and raw variants

11c. Bonus Threshold Targets

Metric	Bonus Threshold	Strategy to Achieve It
LLM-as-a-Judge pass rate	≥ 90%	Tune prompt template; use GraphRAG Global for synthesis questions; verify reference answers are accurate
BERTScore F1 rescaled	≥ 0.55	Use a strong embedding model (deberta-xlarge-mnli); ensure answers are complete, not truncated
BERTScore F1 raw	≥ 0.88	Same tuning; raw score is usually higher — focus on rescaled first
Both thresholds	Maximum bonus	Hitting both unlocks the maximum bonus multiplier in judging

Full Evaluation Run (All Pipelines)

```
def run_full_evaluation(pipelines, qa_pairs):
    report = {}
    for name, pipeline in pipelines.items():
        results = [pipeline.query(qa['question']) for qa in qa_pairs]

        pass_rate, judge_scores = evaluate_pipeline_judge(results, qa_pairs)
        f1_rescaled, f1_raw     = evaluate_pipeline_bertscore(results, qa_pairs)

        report[name] = {
            'judge_pass_rate':  pass_rate,
            'bertscore_rescaled': f1_rescaled,
            'bertscore_raw':    f1_raw,
            'avg_prompt_tokens':   avg([r['prompt_tokens']    for r in results]),
            'avg_completion_tokens':avg([r['completion_tokens'] for r in results]),
            'avg_latency_ms':   avg([r['latency_ms']        for r in results]),
            'avg_cost_usd':     avg([r['cost']              for r in results]),
        }

    # Token reduction (GraphRAG vs Basic RAG)
    base = report['basic_rag']['avg_prompt_tokens'] \
         + report['basic_rag']['avg_completion_tokens']
    graph= report['graphrag']['avg_prompt_tokens'] \
         + report['graphrag']['avg_completion_tokens']
    report['token_reduction_pct'] = (base - graph) / base * 100

    return report
```

Unified evaluation harness — runs all pipelines and collects all metrics

12. Tuning Guide & Hyperparameter Grid

GraphRAG requires iterative tuning — this grid gives you a principled starting point

The hackathon brief explicitly states: *'GraphRAG requires iterative tuning. Teams are expected to adjust key parameters such as chunking strategy, chunk size, hop depth, LLM selection, and prompt design to optimise results.'* Use this grid systematically.

Component	Parameter	Start	Search Range	Optimise For
Corpus prep	chunk_size	384 tok	256 / 384 / 512	Recall vs token count
Corpus prep	chunk_overlap	64 tok	32 / 64 / 96	Boundary coverage
Basic RAG	top_k (dense)	8	5 / 8 / 10 / 12	Recall vs prompt size
GraphRAG	top_k (seed)	5	3 / 5 / 8	Starting breadth
GraphRAG	num_hops	2	1 / 2 / 3	Multi-hop depth (watch token blow-up at 3)
GraphRAG	community_level	2	1 / 2 / 3	Summary granularity
GraphRAG	max_context_tokens	3000	2000 / 3000 / 4000	Token budget hard cap
GraphRAG	retriever_type	Hybrid	Hybrid/Community/Sibling/Local	Query type match
Routing	graph_threshold	0.5	0.3 / 0.5 / 0.7	When to activate graph vs dense
Evaluation	judge_order	Random	Always randomise order	Reduce LLM judge bias

Tuning Protocol

- Start with chunk size — it affects both pipelines and the graph structure equally
- Fix chunk size, then tune num_hops — watch token counts at num_hops=3
- Tune community_level only after hop depth is stable
- Set max_context_tokens as a safety net to prevent prompt overflow
- Tune retriever_type per query stratum (local=entity, global=community, multihop=hybrid)
- Run full evaluation after each change — track all 4 metrics, not just accuracy

■ Do NOT tune on your test questions. Split your question bank: 10 for development/tuning, 10+ for final evaluation. Overfitting to test questions invalidates your benchmark report.

13. Required Deliverables Checklist

All 8 required items — submit via Unstop before the deadline

All submissions go through **Unstop** directly. Upload everything via your Unstop submission form before the deadline.

#	Deliverable	What to Include	Who Needs It
1	**Architecture Diagram**	Visual showing all 3 pipelines, TigerGraph graph layer, evaluation harness, dashboard	*All teams*
2	**Comparison Dashboard**	Interactive page: 1 query in → 3 pipelines run → side-by-side responses + 4 metrics	*All teams*
3	**Benchmark Report**	Tokens, cost, latency, BERTScore, judge pass rate per pipeline. Per query + aggregate.	*All teams*
4	**Demo Video**	5–7 minute walkthrough: show dashboard, run live queries, explain results, narrate graph insight	*All teams*
5	**Public GitHub Repo**	Full source built on TigerGraph GraphRAG repo. README with setup, dataset, run instructions.	*All teams*
6	**Blog Post**	Technical write-up on Medium, Hashnode, or Dev.to. Cover problem, approach, results, lessons.	*All teams*
7	**Social Media Post**	LinkedIn or Twitter post. Tag @TigerGraph. Use #GraphRAGInferenceHackathon	*All teams*
8	**Product Feedback Interview**	Short interview with TigerGraph product team. What worked, what didn't, what would improve GraphRAG.	*Top 5–10 only*

■ GitHub repo TIP: Organise as a mono-repo with /pipeline1_llm, /pipeline2_rag, /pipeline3_graphrag, /dashboard, /evaluation, /data directories. Include a Makefile or run_all.sh that reproduces your full benchmark with one command.

14. Literature References & Further Reading

Key papers that informed this solution — all directly relevant to the hackathon objectives

[1] **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks**
Lewis et al., NeurIPS 2020
Foundational RAG paper. The Pipeline 2 baseline descends from this architecture.
https://arxiv.org/abs/2005.11401

[2] **From Local to Global: A Graph RAG Approach to Query-Focused Summarization**
Edge et al., arXiv 2024
Introduces entity graph + community summaries for corpus-wide synthesis. Direct inspiration for TigerGraph GraphRAG.
https://arxiv.org/abs/2404.16130

[3] **RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval**
Sarthi et al., ICLR 2024
Hierarchical tree retrieval. Best non-graph comparison point for global queries.
https://openreview.net/forum?id=GN921JHCRw

[4] **HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models**
Gutiérrez et al., NeurIPS 2024
Entity graph + Personalized PageRank. 10–20× cheaper than iterative retrieval while matching accuracy.
https://proceedings.neurips.cc/paper_files/paper/2024/hash/6ddc001d07ca4f319af96a3024f6dbd1

[5] **LightRAG: Simple and Fast Retrieval-Augmented Generation**
Guo et al., EMNLP 2025
Lightweight graph with incremental updates. Good reference for deployment efficiency.
https://aclanthology.org/2025.findings-emnlp.568/

[6] **PathRAG: Pruning Graph-based Retrieval Augmented Generation with Relational Paths**
Chen et al., AAAI 2026
Flow-based path pruning. Directly informs Section 8c of this guide. Token-efficient GraphRAG.
https://ojs.aaai.org/index.php/AAAI/article/view/40268

[7] **You Don't Need Pre-Built Graphs for RAG (LogicRAG)**
Chen et al., AAAI 2026
On-the-fly query DAG. Useful for dynamic corpora and reducing indexing overhead.
https://ojs.aaai.org/index.php/AAAI/article/view/40278

[8] **LinearRAG: Linear Graph Retrieval Augmented Generation on Large-scale Corpora**
Zhuang et al., ICLR 2026
Relation-free Tri-Graph. Best reference if graph construction LLM cost is a bottleneck.
https://openreview.net/forum?id=mCtfkypdm6

[9] **Use Graph When It Needs: EA-GraphRAG**
Dong et al., arXiv 2026
Query-complexity router between dense RAG and GraphRAG. Directly informs Section 9 routing algorithm.
https://arxiv.org/abs/2602.03578

[10] **RAG vs. GraphRAG: A Systematic Evaluation and Key Insights**
Han et al., arXiv 2025
Standardised comparison. Essential reading for fair benchmarking methodology and judge bias.
https://arxiv.org/abs/2502.11371

[11] **When to use Graphs in RAG: A Comprehensive Analysis**
Xiang et al., ICLR 2026
Decision paper: graphs help for hierarchical knowledge and deep contextual reasoning, not always otherwise.
https://openreview.net/forum?id=i9q9xDMjG7

[12] **Graph Retrieval-Augmented Generation: A Survey**
Peng et al., ACM TOIS 2025
Best formal taxonomy of the entire GraphRAG design space.
https://arxiv.org/abs/2408.08921

Important Links: TigerGraph GraphRAG Repo: github.com/tigergraph/graphrag | TigerGraph MCP: github.com/tigergraph/tigergraph-mcp | TigerGraph Savanna: tgcloud.io | Docs: docs.tigergraph.com | Discord: discord.gg/4cc7SNqRf

Build it. Benchmark it. Prove graph beats tokens. ■